23. Construct a C Program to Implement First Fit Algorithm of Memory Management

---

Code - Part 1 (Lines 1-35) -- Memory Initialization & Display

```c
1  #include <stdio.h>
2
3  #define MAX_MEMORY 1000
4
5  int memory[MAX_MEMORY];
6
7  // Initialize memory
8  void initializeMemory() {
9      for (int i = 0; i < MAX_MEMORY; i++) {
10         memory[i] = -1;
11     }
12 }
13
14 // Display memory
15 void displayMemory() {
16     int i, j;
17
18     printf("\nMemory Status:\n");
19
20     for (i = 0; i < MAX_MEMORY; i++) {
21         if (memory[i] == -1) {
22             j = i;
23
24             while (j < MAX_MEMORY && memory[j] == -1) {
25                 j++;
26             }
27
28             printf("Free memory block %d-%d\n", i, j - 1);
29             i = j - 1;
30         }
31     }
32 }
33
34 // First-Fit Allocation
35 void allocateMemory(int processId, int size) {
```

Code - Part 2 (Lines 36-70) -- First-Fit Allocation Logic

```
36
37      int start = -1;
38      int blockSize = 0;
39
40      for (int i = 0; i < MAX_MEMORY; i++) {
41
42          if (memory[i] == -1) {
43
44              if (blockSize == 0) {
45                  start = i;
46              }
47
48              blockSize++;
49
50              if (blockSize >= size) {
51                  break;
52              }
53
54          } else {
55              blockSize = 0;
56          }
57      }
58
59      if (blockSize >= size) {
60          for (int i = start; i < start + size; i++) {
61              memory[i] = processId;
62          }
63
64          printf("Allocated memory block %d-%d to Process %d\n",
65                 start, start + size - 1, processId);
66      } else {
67          printf("Allocation failed for Process %d\n", processId);
68      }
69  }
70
```

Code - Part 3 (Lines 71-100) -- Deallocation & Main Function

```
71  // Deallocate
72  void deallocateMemory(int processId) {
73      for (int i = 0; i < MAX_MEMORY; i++) {
74          if (memory[i] == processId) {
75              memory[i] = -1;
76          }
77      }
78
79      printf("Memory released by Process %d\n", processId);
80  }
81
82  int main() {
83
84      initializeMemory();
85      displayMemory();
86
87      allocateMemory(1, 200);
88      displayMemory();
89
90      allocateMemory(2, 300);
91      displayMemory();
92
93      deallocateMemory(1);
94      displayMemory();
95
96      allocateMemory(3, 400);
97      displayMemory();
98
99      return 0;
100 }
```

Output

```
Memory Status:
Free memory block 0-999
Allocated memory block 0-199 to Process 1

Memory Status:
Free memory block 200-999
Allocated memory block 200-499 to Process 2

Memory Status:
Free memory block 500-999
Memory released by Process 1

Memory Status:
Free memory block 0-199
Free memory block 500-999
Allocated memory block 500-899 to Process 3

Memory Status:
Free memory block 0-199
Free memory block 900-999


...Program finished with exit code 0
Press ENTER to exit console.
```